Master 1 - Distributed Artificial Intelligence

Web Programming

MATHÉMATIQUES ET INFORMATIQUE
Sciences
Université Paris Cité

Project Report

EventHub : Event and Participant Management System

Aaron AIDOUDI

Academic Year 2025-2026

Table of contents

1. Introduction 3
2. General Architecture 3
3. Data Model 4
4. Django Backend 4
4.1 API Endpoints 4
4.2 Serializers and Business Validation 5
4.3 Permissions and Authentication 5
4.4 Filtering 5
4.5 API Documentation 6
5. Node.js Backend and Comparative Analysis 6
5.1 Node.js / Express Implementation 6
5.2 Django vs Node.js - Comparative Analysis 6
6. React Frontend 7
6.1 Component Architecture 7
6.2 Authentication and Route Protection 8
6.3 Role-based Interface 8
6.4 Design System 8
7. Deployment 9
7.1 Backend and Database (Railway) 9
7.2 Frontend (Vercel) 9
8. Challenges and Solutions 9
9. Conclusion 10

1. Introduction

This report presents the design, implementation, and deployment of **EventHub**, a **full-stack** web application for **managing events and participants**. The project was carried out as part of the Web Programming course and covers the full development lifecycle, from relational data modeling to production deployment.

EventHub allows users to create and manage events, register participants, and control access based on user roles. The application is publicly accessible at ***eventhub-iota.vercel.app*** and the source code is available at the project's Git repository *github.com/aaidoudi91/EventHub*, organized into three distinct directories: *backend-django*, *backend-node*, and *frontend*.

The technical stack was chosen to demonstrate the integration of multiple modern web technologies. The primary backend is built with **Python/Django** and **Django REST Framework**, exposing a complete **REST API** consumed by a **React** single-page application built with **Vite**. A secondary backend implemented with **Node.js** and **Express** serves as a comparative study alongside the Django implementation. Both backends rely on **PostgreSQL** as the database engine, and authentication is handled via **JWT** tokens using the **SimpleJWT** library.

2. General Architecture

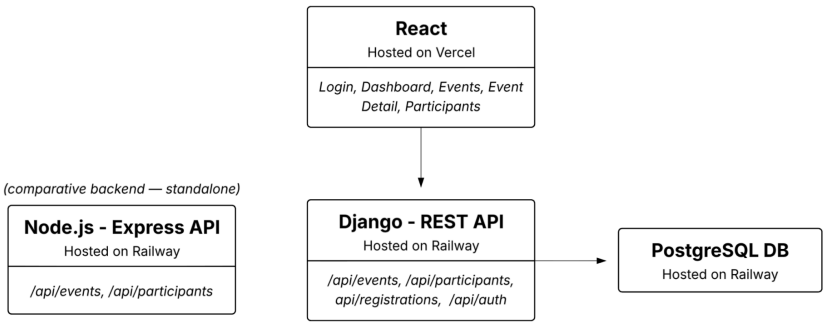
EventHub follows a classic **three-tier architecture**, separating the **presentation layer**, the **business** logic layer, and the **data** layer. The React frontend communicates exclusively with the Django backend through **HTTP** requests to a **REST API**. The Node.js backend operates independently, serving solely as a **comparative** implementation rather than a component of the production system.

The **Django backend** acts as the central hub of the application. It handles all business logic, enforces validation rules, manages authentication, and serves data to the frontend through a set of structured REST endpoints. All API responses are formatted as JSON, and every protected route requires a valid JWT token passed in the Authorization header of each request.

The **frontend** is a **Single Page Application** that never reloads the page during navigation. React Router handles all client-side routing, and Axios is used to communicate with the API. The authentication token is stored in the browser's localStorage and automatically injected into every outgoing request via an Axios interceptor. Route protection is enforced at the component level, redirecting unauthenticated users to the login page before any API call is made.

The **production infrastructure** is split across two platforms. The Django backend and PostgreSQL database are hosted on **Railway**, while the React frontend is deployed on **Vercel**. Environment variables are used on both platforms to separate development and production configurations, ensuring that no sensitive values such as secret keys or database credentials are stored in the codebase.

The diagram below illustrates the overall **architecture**:



3. Data Model

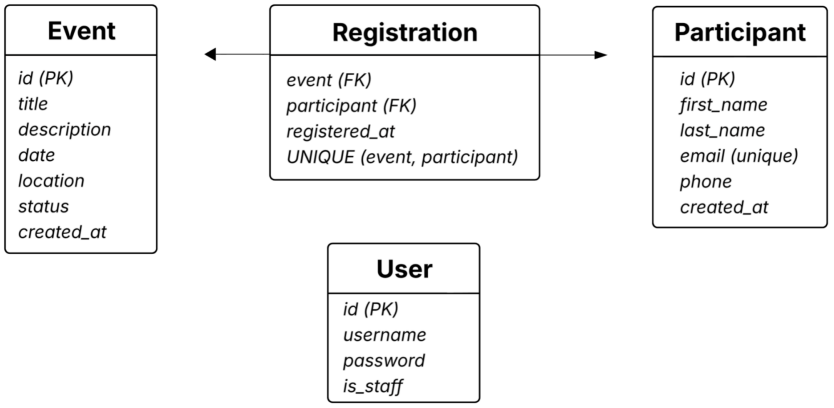
The data model is built around three core entities: **Event**, **Participant**, and **Registration**. A fourth entity, Django's built-in **User** model, handles authentication and role management without requiring a custom implementation.

The **Event** model represents a **scheduled occasion** with a *title*, *description*, *date*, *location*, and a *status* field restricted to three possible values: *open*, *closed*, and *cancelled*. The status field plays a central role in the business logic, as registrations are only permitted for events whose status is *open*. The *created_at* field is automatically populated at creation and is used by the frontend to display the most recently added participants on the dashboard.

The **Participant** model stores the **personal information** of individuals who can be registered to events: *first_name*, *last_name*, a unique *email* address, and an optional *phone* number. The uniqueness constraint on the email field is enforced at the database level, preventing duplicate profiles from being created regardless of how the API is called.

The **Registration** model is the central piece of the relational design. Rather than using Django's implicit *ManyToManyField*, a dedicated model was implemented with explicit foreign keys to both Event and Participant. This design choice makes the relationship a first-class entity in the API, allowing registrations to be listed, created, and deleted independently. The *unique_together* constraint on the *(event, participant)* pair enforces at the database level that a participant cannot be registered twice for the same event, complementing the validation already performed at the serializer level.

The diagram below illustrates the relationships between the four entities:



4. Django Backend

The Django backend serves as the primary **API** of the application. It is built with **Django 5** and **Django REST Framework**, and exposes a complete **REST API** covering all entities defined in the data model. The project is structured around a single Django application named *events*, which contains all models, serializers, views, permissions, filters, and URL configuration.

4.1 API Endpoints

The routing is handled by DRF's **DefaultRouter**, which automatically generates the standard REST endpoints for each registered ViewSet. The following routes are exposed under the */api/* prefix:

Method	Endpoint	Description
POST	*/auth/login/*	Obtain JWT token pair
POST	*/auth/refresh/*	Refresh access token
GET / POST	*/events/*	List all events / Create
GET / PUT / DELETE	*/events/{id}/*	Retrieve / Update / Delete
GET / POST	*/participants/*	List all participants / Create
GET / PUT / DELETE	*/participants/{id}/*	Retrieve / Update / Delete
GET / POST	*/registrations/*	List all / Register participant
GET / DELETE	*/registrations/{id}/*	Retrieve / Remove registration
GET	*/docs/*	Swagger UI (drf-spectacular)

4.2 Serializers and Business Validation

Serializers serve two purposes in this project: data transformation and business rule enforcement.

The *RegistrationSerializer* implements two validation rules in its *validate()* method. First, it checks whether a registration already exists for the given *(event, participant)* pair and raises a 400 Bad Request if so, preventing duplicate registrations before the database constraint is even reached. Second, it verifies that the target event has a status of open, rejecting any attempt to register a participant to a closed or cancelled event.
The *RegistrationViewSet* uses two different serializers depending on the action being performed. Read operations *(list, retrieve)* use a *RegistrationDetailSerializer* that nests the full *Event* and *Participant* objects, giving the frontend all the information it needs in a single request. Write operations use the standard *RegistrationSerializer* that expects only foreign key IDs, keeping the creation payload minimal and unambiguous.

4.3 Permissions and Authentication

Authentication is handled by **SimpleJWT**, which issues a short-lived access token valid for one hour and a long-lived refresh token. The JWT payload was extended with a custom serializer to include the *is_staff* field, allowing the React frontend to determine the user's role directly from the token without making an additional API call.

Access control is enforced by a custom permission class, *IsAdminOrReadOnly*, which permits all authenticated users to perform safe HTTP methods (*GET*, *HEAD*, *OPTIONS*) while restricting write operations (*POST*, *PUT*, *PATCH*, *DELETE*) exclusively to users with *is_staff=True*. This single class, applied uniformly across all three ViewSets, implements the complete role-based access control required by the project.

4.4 Filtering

The **EventViewSet** supports filtering through django-filter. The *EventFilter* class exposes three query parameters: *status* for exact match filtering, and *date_from* / *date_to* for range-based date

filtering using gte and lte lookups respectively. These parameters can be combined freely, allowing the frontend to query for example all open events within a specific date range in a single request.

4.5 API Documentation

The API is fully documented using **drf-spectacular**, which introspects the ViewSets and serializers at runtime to generate an OpenAPI 3.0 schema. The resulting Swagger UI is accessible at */api/docs/* and provides an interactive interface to explore and test all endpoints, including authentication via JWT.

5. Node.js Backend and Comparative Analysis

5.1 Node.js / Express Implementation

The **Node.js backend** is a simplified implementation of the API covering the Event and Participant entities. It is built with **Express** as the web framework and **Sequelize** as the ORM. The project follows a straightforward layered structure, separating database configuration, model definitions, route handlers, and error middleware into distinct files.

Each entity exposes the five standard **CRUD routes**: list all, retrieve one, create, update, and delete. The *Event* routes additionally support filtering by status via a query parameter, mirroring the basic filtering capability of the Django backend. Model definitions in Sequelize include field-level validation, such as email format checking on the *Participant* model and an *ENUM* constraint on the *Event* status field, ensuring data integrity at the application level.

Error handling is centralized in a single middleware function registered at the end of the Express middleware chain. Any error thrown or passed via *next(err)* within a route handler is caught by this middleware, which returns a structured JSON response with the appropriate HTTP status code. This approach avoids duplicating error handling logic across individual route handlers and ensures a consistent error response format throughout the API.

5.2 Django vs Node.js - Comparative Analysis

Implementing the same API with two different stacks provides a concrete basis for comparison across five dimensions.

i. Project Structure: Django imposes a well-defined project structure from the outset. Applications, models, views, serializers, and URL configurations each have a designated place, and the framework enforces this organization through its conventions. Node.js with Express imposes nothing. The developer is entirely responsible for deciding how to organize the codebase, which offers flexibility but also introduces the risk of inconsistency, especially as the project grows.

ii. Architectural Philosophy: Django is a “batteries included” framework. An ORM, an admin interface, a migration system, an authentication backend, and a permission system are all available out of the box without installing a single additional package. Node.js follows the opposite philosophy: the core framework is deliberately minimal, and every capability beyond basic HTTP routing must be sourced from the npm ecosystem. This means choosing an ORM (Sequelize, Prisma, TypeORM), a validation library, an authentication strategy, and assembling them into a coherent system. The resulting stack can be more precisely tailored to a specific use case, but the assembly work is entirely the developer's responsibility.

iii. Development Complexity: For a standard CRUD API with authentication and relational data, Django requires significantly less code than an equivalent Express implementation. DRF's ViewSets and serializers abstract away the repetitive logic of request parsing, validation, and response formatting. The same endpoints in Express require writing each handler explicitly, managing database calls directly, and handling errors at every step. For this project specifically, the Django implementation took considerably less time to write while covering more features, including a richer permission system, JWT customization, and automatic API documentation.

iv. Scalability: Node.js has a well-established advantage in scenarios involving high concurrency and I/O-bound workloads. Its event-driven, non-blocking architecture allows a single process to handle a large number of simultaneous connections efficiently, making it particularly suited for real-time applications, WebSocket servers, or high-throughput APIs. Django's synchronous request-handling model, while capable of handling significant load with proper deployment configuration, is architecturally less suited to these scenarios. For a data-heavy application with complex queries and business logic, however, Django's mature ORM and straightforward execution model often result in more maintainable code.

v. Ecosystem: Both ecosystems are mature and actively maintained. npm offers the largest package registry in existence, giving Node.js developers access to an enormous range of libraries for virtually any use case. Python's ecosystem, while smaller, is exceptionally strong in domains relevant to web backends: data processing, scientific computing, and machine learning. For a team that already uses Python for data science or AI work, Django represents a natural choice, as it allows sharing code, tooling, and expertise across the backend and other parts of the stack.

In summary, **Django** proved to be the **more productive choice** for this project. The combination of DRF, built-in authentication, and automatic admin interface reduced the time spent on infrastructure and allowed more focus on business logic. Node.js remains a compelling option for use cases where sharing code between the client and server or high-concurrency I/O performance are primary concerns.

6. React Frontend

The frontend is a **Single Page Application** built with **React 18** and **Vite**. It communicates exclusively with the Django backend and covers all functional requirements defined in the project: event management, participant management, registration handling, authentication, and role-based access control. The application supports both a light and dark theme, with the user's preference persisted in *localStorage* across sessions.

6.1 Component Architecture

The source code is organized into five directories: *api/*, *components/*, *context/*, *pages/*, *styles/*. This separation reflects the different responsibilities within the frontend: API communication, reusable UI elements, global state, page-level views, and shared design constants respectively.

The application is structured around some **reusable components** like *Navbar*, *ProtectedRoute*, *Spinner*, and *StatusBadge*; and the **pages** *LoginPage*, *DashboardPage*, *EventsPage*, *EventDetailPage*, and *ParticipantsPage*. The reusable components handle respectively top-level navigation, route protection for unauthenticated users, loading feedback, and consistent status display across the interface. Each page is self-contained and corresponds to a distinct functional area of the application. All components are functional and rely exclusively on **React Hooks**. ***useState*** manages local UI state such as form values, loading flags, and error messages. ***useEffect*** triggers data fetching

on component mount and whenever filter values change. ***useContext*** provides access to the two global contexts described below.

6.2 Authentication and Route Protection

Authentication state is managed by ***AuthContext***, a React context that exposes the current user object, a login function, and a logout function. On successful login, the Django backend returns an access token and a refresh token. Both are stored in *localStorage*, and the user's username and role are extracted by decoding the JWT payload client-side using atob, avoiding an additional API call. The *is_staff* field included in the custom JWT payload determines whether the user is treated as an admin or a viewer throughout the application.

All API requests are made through a single **Axios** instance configured in *api/axios.js*. An Axios request interceptor reads the access token from localStorage on every outgoing request and injects it into the *Authorization* header automatically, ensuring that no individual component needs to handle token attachment manually.

Route protection is implemented via the ***ProtectedRoute*** component, which wraps every page except the login page. If no user is found in ***AuthContext***, the component renders a redirect to */login* before the page content or any API call is executed.

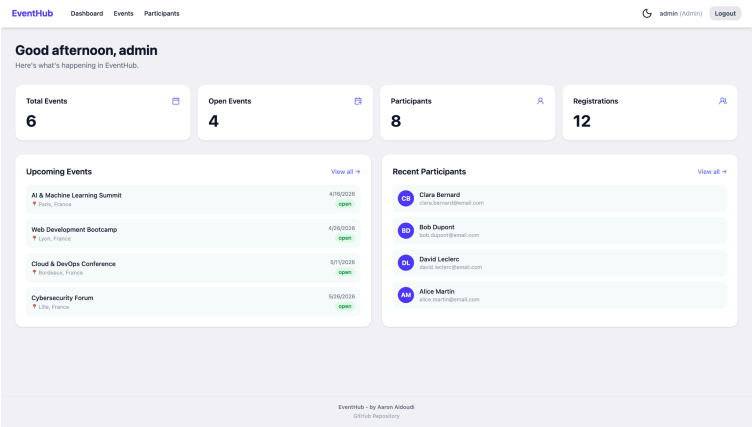
6.3 Role-based Interface

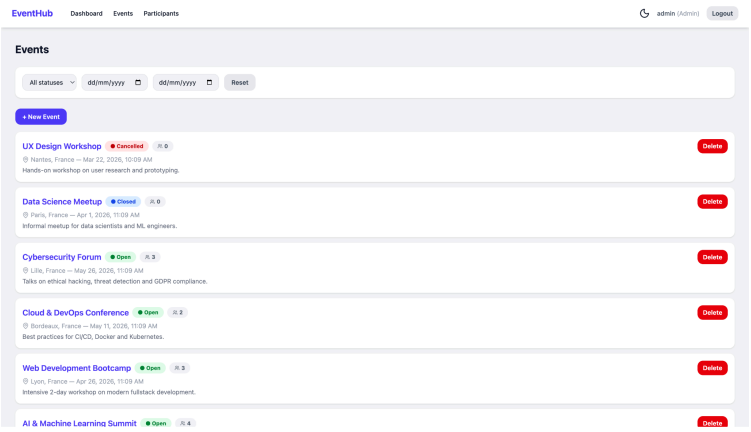
The **role distinction** between admin and viewer is reflected throughout the interface. Buttons that trigger write operations (creating, editing, deleting events or participants, registering or removing participants from an event) are conditionally rendered based on the *user.isAdmin* boolean available from *AuthContext*. A viewer navigating the application sees an identical layout but with all modification controls absent, making the read-only nature of their access immediately clear without requiring any additional explanation.

6.4 Design System

All UI constants are centralized in *src/styles/ui.js*, which exports named class strings for each button variant and status badge. This ensures visual consistency across all pages without duplicating style definitions, a button of a given type always looks identical regardless of where it appears in the application.

The styling is implemented with **Tailwind CSS**, configured via the Vite plugin. The dark mode is toggled by adding or removing the dark class on the root html element, managed by *ThemeContext*. Every component uses paired light and dark class variants, ensuring a consistent appearance in both modes.





7. Deployment

The application is deployed across two platforms: **Railway** for all backend services and the database, and **Vercel** for the React frontend. This split reflects the natural division between server-side and static workloads, and takes advantage of the free tiers offered by both platforms.

7.1 Backend and Database (Railway)

Railway hosts two services within a single project: the **PostgreSQL** database and the **Django** backend. Each service is deployed directly from the Git repository by specifying the relevant root directory, allowing Railway to detect the appropriate runtime and install dependencies automatically.

The Django backend is served in production by **Gunicorn**, a production-grade WSGI server. Static files are handled by **WhiteNoise**, which compresses and serves them directly from the Django process without requiring a separate static file server. This approach means the application uses SQLite locally for development and switches to PostgreSQL in production transparently, without any code change.

After each deployment, database migrations are applied and the seed command is run to populate the database with realistic sample data, ensuring the application is immediately usable for demonstration purposes.

7.2 Frontend (Vercel)

The **React** frontend is deployed on **Vercel** with the root directory set to frontend. Vercel runs npm run build, which produces an optimized static bundle in the dist directory. The *VITE_API_URL* environment variable is set to the Railway Django service URL, and Vite injects it into the bundle at build time, ensuring all API requests target the correct production endpoint.

Since EventHub is a Single Page Application, all routing is handled client-side by React Router. A *vercel.json* configuration file at the root of the frontend directory instructs Vercel to rewrite all incoming requests to index.html, preventing 404 errors when users navigate directly to a URL or refresh the page on any route other than the root.

The production application is publicly accessible at *eventhub-iota.vercel.app*

8. Challenges and Solutions

Several technical difficulties arose during the development and deployment of the project.

i. JWT payload and role detection: The default SimpleJWT configuration does not include any application-specific fields in the token payload. The frontend needed to know whether the authenticated user was an admin or a viewer immediately after login, without making an additional API call. The solution was to extend the *TokenObtainPairSerializer* with a custom subclass that adds the *is_staff* boolean to the token payload during generation. The corresponding view was then registered in the URL configuration, replacing the default login endpoint transparently.

ii. CORS configuration between Railway and Vercel: Cross-origin requests from the Vercel frontend to the Railway backend were initially blocked by the browser due to missing CORS headers. The django-cors-headers middleware was already installed, but the *CORS_ALLOWED_ORIGINS* setting needed to contain the exact Vercel deployment URL.

iii. React Router and Vercel's static routing: After deployment, refreshing the browser on any page other than the root returned a 404 error. Vercel treats the frontend as a collection of static files and attempts to resolve each URL to a physical file on disk. Since React Router handles all navigation client-side, no such files exist beyond index.html. Adding a *vercel.json* file with a single rewrite rule redirecting all requests to *index.html* resolved the issue completely, restoring correct navigation behavior across all routes.

iv. Date format compatibility between Django and HTML inputs: Django stores and returns datetime values in ISO 8601 format, which includes timezone information such as 2026-04-15T14:00:00Z. The HTML datetime-local input element, used in the event creation and editing forms, expects a truncated format without timezone information. Pre-populating the edit form with a value directly from the API therefore produced a format mismatch that left the input field empty. Slicing the first 16 characters of the datetime string with *.slice(0, 16)* before assigning it to the form state resolved the issue and ensured the field was correctly pre-filled when opening the edit form.

9. Conclusion

EventHub demonstrates the integration of multiple modern web technologies into a coherent, production-ready application. The project covers the full development lifecycle, from relational data modeling and REST API design to frontend development and cloud deployment, addressing every functional requirement defined in the project specification.

The **Django backend** proved to be a robust and productive foundation for the API, with Django REST Framework significantly reducing the boilerplate typically associated with building a secure, well-structured REST interface. The **React frontend** delivers a complete user experience across all defined pages, with consistent design, role-based access control, and proper handling of loading and error states throughout. The **Node.js** implementation, while limited in scope, provided a meaningful basis for a structured comparison between two fundamentally different approaches to backend development.

The deployment on **Railway** and **Vercel** demonstrated that a full-stack application with separate backend and frontend services can be brought to production with minimal infrastructure configuration, provided that environment variables, CORS policies, and build pipelines are properly managed. The application is publicly accessible, stable, and populated with realistic sample data, making it immediately ready for demonstration.